

— A UNIFIED ML FRAMEWORK FOR RUST

Ten crates, one lifecycle.

the trade of assembling ten crates into one working machine

scikit-learn stops at `.predict()`. **Millwright** ends at a served, drift-monitored ONNX model — and treats every step in between as one composable pipeline.

fit · transform · predict

one **Frame**, every backend

train → **ONNX** → serve → watch

capabilities are **cargo features**

Rust core · **Python** API

— THE IDEA

Ship the assembly, not the parts.

You already built the ecosystem — `plotters-statistical`, `model-selection-rs`, `imbalance-rs`, `regression-diagnostics`, `hyperopt-rs`, `shap-rs`, `driftwatch`, `onnx-export-rs`, `incremental-rs`, `chronos-ts` — plus the established `linfa` / `smartcore` / `polars` stack.

Today a practitioner assembles those by hand: mismatched array types, no shared `fit` / `transform` contract, and manual glue from train to export to serve to monitor. **Millwright is that assembly** — one data model, one trait contract, one pipeline, feature-gated backends. The parts stay independently useful; Millwright is the tradesman that assembles them into one coherent machine — and keeps it running.

— THE THROUGH-LINE

The whole lifecycle, as pipeline stages.

Each stage is a first-class step powered by a crate you already own. Exploration sits up front — the Rust answer to `pandas-profiling` — and the last two stages are where Millwright runs past scikit-learn.

01 · INGEST

Load & frame

`polars` → **Frame**

CSV/Parquet/Arrow into one columnar container with a schema.

02 · EXPLORE

Profile the data

`plotters-statistical` · `polars`

One call → summary stats, distributions, missingness, correlation, outliers, target relationship.

03 · PREP

Preprocess

`imbalance-rs`

Impute · encode · scale · SMOTE — as pipeline transformers.

05 · FIT

Train

`linfa` · `smartcore`

Any backend model, behind one `Estimator` contract.

07 · EXPLAIN

Interpret

`shap-rs`

SHAP values & permutation importance on the fitted pipeline.

09 · OPERATE

Serve & monitor

`axum` · `driftwatch`

A `/predict` endpoint with live PSI drift on the request stream.

04 · SELECT

Search & validate

`model-selection-rs` · `hyperopt-rs`

Stratified CV, grid / random / Bayesian search over the pipeline.

06 · ASSESS

Evaluate

`regression-diagnostics`

Metrics, residual & calibration diagnostics, plotters-statistical reports.

08 · EXPORT

To ONNX

`onnx-export-rs`

One portable artifact — train in Rust, serve anywhere.

— ARCHITECTURE

A thin contract over proven engines.

Four layers. The public API never names a specific ndarray version, and the core stays small — every engine plugs in through the same four traits.

▲ you write this

Fluent API

MILLWRIGHT::PRELUDE

Profile

Pipeline

GridSearch

Explainer

Server

DriftMonitor

Registry

Core contract

THE FRAMEWORK

Frame / Dataset

trait Estimator

trait Transformer

trait Predictor

trait ProbaPredictor

ColumnTransformer

Backend adapters

#[CFG(FEATURE)]

`smartcore` ⇄

`linfa` ⇄

`chronos-ts` ⇄

`incremental-rs` ⇄

`tract (onnx infer)`

Engines

YOUR CRATES + THE STACK

`linfa`

`smartcore`

`polars`

`model-selection-rs`

`hyperopt-rs`

`imbalance-rs`

`shap-rs`

`regression-diagnostics`

`onnx-export-rs`

`driftwatch`

`chronos-ts`

`plotters-statistical`

▼ these keep shipping as standalone crates

The four traits

Object-safe so a `Pipeline` can hold a heterogeneous `Vec<Box<dyn ...>>`.
Everything composes because everything speaks the same contract.

- **Estimator** — `fit(&Dataset) → Fitted`
- **Transformer** — `transform(&Frame) → Frame`
- **Predictor** — `predict(&Frame) → Array`
- **ProbaPredictor** — `predict_proba(&Frame) → Frame`

Params by path

Pipeline steps are addressable by name, so a search can tune any parameter anywhere in the chain — the scikit-learn `"step__param"` convention.

```
// tune the forest inside a 4-step pipeline
grid! {
  "scale__with_mean" ⇒ [true, false],
  "rf__max_depth"    ⇒ [4, 8, 16],
  "rf__n_trees"      ⇒ [100, 300],
}
```

— THE HARD PROBLEM · SOLVED BY DESIGN

Two ndarray worlds, one Frame.

linfa pins ndarray 0.15; your newer crates use 0.16; smartcore has its own DenseMatrix. These cannot meet in one function signature. This is the reason a unified framework doesn't exist yet — so it's the first thing the design settles.

The framework owns the boundary type

Millwright's `Frame` is a contiguous `f64` buffer + schema + optional target. The *public* API only ever speaks `Frame` — users are never locked to a version, exactly how pandas/NumPy sit under scikit-learn.

```
Frame { buf: Vec<f64>, shape, cols, target }
```

Adapters convert at the edge only

Each backend adapter converts `Frame` ⇄ its native type *inside* the adapter — an O(n) copy over a row-major buffer, with a zero-copy fast path where layout and version already agree. The version war never reaches user code.

```
frame.as_nd15() · as_nd16() · as_dense() → used only in
adapters
```

— WHAT IT FEELS LIKE

Data in, monitored service out.

One fluent chain that would today be a dozen crates and a hundred lines of glue.
This is the whole pitch in one screen.

```
use millwright::prelude::*;

// 0 - look before you leap: one call profiles the whole frame to HTML
Profile::of(&train)?.to_html("eda_report.html"?); // plotters-statistical + polars

// 1 - compose a pipeline: preprocessing + a model, one object
let pipe = Pipeline::new()
  .step("impute", SimpleImputer::median())
  .step("encode", OneHot::infer())
  .step("scale", StandardScaler::new())
  .balance(Smote::default()) // imbalance-rs · train-time only
  .estimator("rf", RandomForest::new());

// 2 - search & cross-validate the whole pipeline
let model = GridSearch::new(pipe, grid! { "rf__max_depth" ⇒ [4, 8, 16] })
  .cv(StratifiedKFold::new(5)) // model-selection-rs
  .scoring(Metric::F1)
  .fit(&train)?;
```

```
// 3 - assess & explain
let report = model.evaluate(&test)?;           // metrics + regression-diagnostics
let shap   = model.explain(Explainer::kernel(), &test)?; // shap-rs

// 4 - ship it: one ONNX artifact, served, with drift on every request
model.export_onnx("churn.onnx"?;           // onnx-export-rs

Server::from_onnx("churn.onnx")           // tract inference
  .with_monitor(DriftMonitor::psi(&train)) // driftwatch
  .route("/predict")
  .serve("0.0.0.0:8080").await?;          // axum
```

The `timeseries` feature swaps the estimator for a `chronos-ts` auto-ARIMA forecaster behind the same `fit / predict` contract; `incremental` swaps `.fit()` for `.partial_fit()` over batches that never fully load into memory.

— SPOTLIGHT · EXPLORATION

One call profiles the data — and drafts the pipeline.

`Profile::of(&frame)` is the Rust answer to ydata-profiling, with a twist only a framework that owns the whole lifecycle can pull off: it returns a *typed* analysis (not just an HTML blob), renders a shareable report, and hands back a **suggested preprocessing pipeline** to start from.

What it computes

- OVERVIEW**
Shape, dtypes, memory, duplicate rows, and overall missingness at a glance.
- PER-COLUMN**
Numeric: mean/std, quartiles, skew & kurtosis, zeros, distinct, histogram. Categorical: mode, frequencies, cardinality.
- MISSINGNESS**
Per-column nulls, a missingness matrix, and whether columns tend to go missing *together*.
- CORRELATIONS**
Pearson & Spearman matrices with high- $|r|$ pairs flagged — an early read on multicollinearity.
- OUTLIERS**
IQR and z-score flags per numeric column, with counts — ready to winsorize or robustly scale.
- TARGET RELATIONSHIP**
Classification: class balance & per-feature split by class. Regression: feature-vs-target strength.

What it returns

Typed fields you can branch on in code — the HTML report is just one renderer over them.

```
struct Profile {
  overview: Overview,           // shape · dtypes · dups · missing
  columns: Vec<ColumnProfile>, // Numeric|Categorical|Datetime
  missingness: Missingness,     // nulls + co-missing map
  correlations: CorrMatrix,     // Pearson+Spearman, flagged
```

Alerts that map to steps

Because Millwright owns EDA *and* the pipeline, every data-quality alert n

ALERT	SUGGESTED STEP
High missingness	<code>SimpleImputer</code>
High-cardinality category	<code>TargetEncoder</code>

```
target:      Option<TargetProfile>,
alerts:      Vec<Alert>,          // the actionable summary
}
```

Renderers: `.to_html(path)` · `.summary()` (text) · `.alerts()` . Scales past memory — `Profile::of` also accepts a lazy/streaming frame via polars.

ALERT	SUGGESTED STEP
Constant / zero-variance	Drop
Correlated pair · $ r > .95$	drop one · flag VIF
Skewed / heavy-tailed	PowerTransform
Class imbalance	Smote
Outliers (IQR)	Winsorize

The loop scikit-learn can't close

scikit-learn profiles nothing and proposes nothing; ydata-profiling profiles but stops at a report. Millwright turns the profile into a running head start:

```
let profile = Profile::of(&train)?;           // full EDA, one call
profile.to_html("eda_report.html"?);         // a shareable report

for alert in profile.alerts() {               // the data-quality summary, typed
  println!("{alert}");
  // income: 12% missing → impute · city: 41 levels → target-encode · target 4:1 → balance
}

// EDA drafts the starting pipeline — you just add the model
let pipe = profile.suggest_pipeline()         // imputers · encoders · scalers · SMOTE
  .estimator("rf", RandomForest::new());
```

— SPOTLIGHT · ENSEMBLES

Combine models — even across backends.

Because every model is a **Predictor**, combining them is just another **Predictor** that holds several — no new machinery. And the trait is backend-agnostic, so a `linfa` model, a `smartcore` forest, and a `chronos-ts` forecaster can sit in one ensemble. scikit-learn can only ensemble scikit-learn.

Three ways to combine

VOTING

Hard (majority) or soft (mean-probability) vote over several fitted models — the quickest lift over any single one.

STACKING

A meta-learner trained on the base models' *out-of-fold* predictions — leak-free, because the CV engine supplies the folds.

BAGGING

Bootstrap-resample, fit a base estimator per sample in parallel, aggregate — and it works for *any* estimator, not just trees.

Composition, not configuration

```
// soft-vote across three different model families → one Predictor
let vote = Voting::soft()
  .add("lr", LogisticRegression::new())
  .add("rf", RandomForest::new())
  .add("svc", Svc::rbf());

// stack: a meta-learner on leak-free out-of-fold base predictions
```

Free by construction

- **No new crate.** Voting, stacking, and bagging are pure composition over the four traits — they live in the core and are always on.
- **Cross-backend.** The unified **Predictor** is what lets a `linfa`, a `smartcore`, and a `chronos-ts` model vote together — the one thing scikit-learn structurally cannot do.

```
let stack = Stacking::meta(LogisticRegression::new())
  .base("rf", RandomForest::new())
  .base("knn", Knn::k(15))
  .cv(StratifiedKFold::new(5)); // model-selection-rs

// bag any estimator, fanned out over rayon
let bag = Bagging::of(Svc::rbf()).n_estimators(50).parallel();

// an ensemble IS an estimator — tune a member straight through it
let model = GridSearch::new(stack, grid! { "rf__max_depth" => [8, 16] })
  .cv(StratifiedKFold::new(5)).fit(&train)?;
```

- **Leak-free stacking.** Out-of-fold predictions come from the same `model-selection-rs` CV engine, so the meta-learner never sees a base model's own training rows.
- **Still just a model.** Ensembles are `Estimator`s — pipeline-able, searchable per member, ONNX-exportable, SHAP-explainable.

Native ensembles — **RandomForest**, **ExtraTrees**, **GradientBoosting** — arrive from the backends as ordinary estimators, tunable and pipeline-able like anything else.

— SPOTLIGHT · AUTOML

The framework, pointed at itself.

Everything above — profiling, preprocessing, cross-validation, hyperparameter search, ensembling — is exactly what an AutoML engine needs. So Millwright's **AutoML** isn't a bolt-on: it's the framework orchestrating its own parts. Point it at a dataset and a budget; get back the best *deployable* pipeline and a leaderboard.

What it searches

PREPROCESSING

Imputation, encoding, scaling strategies — *seeded* by the `Profile`'s alerts, not brute-forced blind.

MODEL ZOO

Linear · KNN · SVM · forests · boosting — across backends, all behind the one `Estimator` contract.

HYPERPARAMETERS

TPE / Bayesian search per candidate (`hyperopt-rs`), every fit scored by the CV engine.

AUTO-ENSEMBLE

Stack the top-*k* candidates into a final blend — the auto-sklearn move, using the ensemble core.

Point it at data, get a pipeline

```
let result = AutoML::classifier()
  .budget(Budget::trials(200)) // or Budget::minutes(10)
  .metric(Metric::F1)
  .cv(StratifiedKFold::new(5))
  .parallel() // search fans out over rayon
  .fit(&train)?;

println!("{}", result.leaderboard()); // ranked pipelines + scores
let best = result.best(); // a normal, fitted Pipeline

// ...and it flows straight into the rest of the lifecycle
best.explain(Explainer::kernel(), &test)?;
best.export_onnx("model.onnx"); // deployable — unlike a TPOT object
```

More than a wrapper

- **Seeded, not blind.** The search starts from `Profile::suggest_pipeline()` — EDA's findings prune the space before a single model is fit.
- **A deployable artifact.** auto-sklearn and TPOT hand you a Python object; Millwright's winner is an ONNX-exportable, servable, monitorable pipeline.
- **Its own parts.** No separate AutoML crate to trust — it reuses `model-selection-rs`, `hyperopt-rs`, and the ensemble core you already use by hand.
- **Budgeted & parallel.** Cap it by trials or wall-clock; candidates evaluate across cores and the leaderboard fills in live.

— SPOTLIGHT · INTEROP

Rust core, Python API.

scikit-learn's users live in Python — so to stand toe to toe, Millwright ships a first-class Python package: the Polars playbook, a Rust engine behind a Pythonic API. Write the pipeline in Python, run it at Rust speed, pass pandas or NumPy straight in, and get an ONNX model out.

The same pipeline, from Python

```
import millwright as mw

train = mw.Frame.from_pandas(df)          # or from_polars / from_numpy

mw.Profile.of(train).to_html("eda.html")  # same EDA, Rust speed

pipe = (mw.Pipeline()
        .step("impute", mw.SimpleImputer.median())
        .step("scale", mw.StandardScaler())
        .estimator("rf", mw.RandomForest()))

model = (mw.GridSearch(pipe, {"rf__max_depth": [4, 8, 16]})
        .cv(mw.StratifiedKFold(5))
        .fit(train))

model.explain(mw.Explainer.kernel(), test) # shap-rs
model.export_onnx("churn.onnx")           # serve anywhere
```

How it fits

- **pyo3 + numpy bindings.** `pip install millwright`; the `Frame` maps to numpy / pandas / polars, zero-copy where the layout already agrees.
- **ONNX both directions.** Consume scikit-learn or PyTorch models (exported to ONNX, run through `tract`) as pipeline steps — and export Millwright pipelines to ONNX for any Python serving stack.
- **One codebase, not a fork.** The Python API is a thin binding over the same Rust traits — no duplicated logic, no drift. It lives behind the `python` feature.
- **Notebook-native both ways.** Usable from Jupyter (Python) for the data-science mainstream, while the guide's evcxr notebooks stay the Rust reference.

— SPOTLIGHT · MLOPS

A model isn't done when it's trained.

The moment a model serves traffic you need to know what produced it — which data, which pipeline, which metrics — and to roll back when something slips. scikit-learn tracks none of that. Millwright's `Registry` versions the whole artifact and closes the loop back to retraining.

What a version records

ARTIFACT

The fitted pipeline *and* its ONNX export, content-addressed so identical models dedupe.

LINEAGE

Data hash, config, random seed, git commit — enough to reproduce the exact model later.

METRICS

Held-out and CV scores travel with the version, so any two are comparable at a glance.

REFERENCE

The training distribution the drift monitor watches live traffic against — not a guess.

Register, serve, roll back

```
// version a trained pipeline - artifact + ONNX + metrics + lineage
let v = Registry::local("./models")
    .register("churn", &model)?
    .tag("prod");                               // a movable pointer
```

Closing the loop

- **Content-addressed.** A version is the hash of its artifact; a tag like `prod` is just a pointer you can move or revert without copying anything.
- **Reproducible.** Data hash + config + seed + commit is enough to rebuild the exact model — the thing "it

```
// serve straight from the registry; the monitor uses the stored reference
Server::from_registry("churn", "prod")
  .with_monitor(DriftMonitor::from_registry(&v))
  .serve("0.0.0.0:8080").await?;

// when drift fires: retrain on the recorded lineage – or revert in one line
Registry::local("./models").rollback("churn", "prod"?;
```

worked yesterday" usually can't.

- **Monitored against truth.** Drift compares live traffic to the version's *own* stored training distribution, so alerts mean something.
- **The retrain loop.** When drift fires, the lineage is right there to retrain on fresh data — and the previous version is one **rollback** away. That's the loop scikit-learn leaves as homework.

— "AS FEATURES" – EXACTLY AS YOU ASKED

Pull only what you need.

Every capability is a cargo feature over one crate. `default` is a lean, useful core; `full` lights up the whole lifecycle. A serving binary need never compile SHAP; a notebook need never compile axum.

FEATURE	CRATE	ADDS
smartcore-backend DEFAULT	smartcore	KNN · NB · SVM · trees · forests · linear
preprocessing DEFAULT	imbalance-rs	impute · scale · encode · SMOTE transformers
model-selection DEFAULT	model-selection-rs	stratified/group/time CV · grid · random
ensemble DEFAULT	core	voting · stacking · bagging meta-estimators — compose any Predictors, across backends
eda	plotters-statistical · polars	automated Profile report: stats · distributions · missingness · correlation · outliers
linfa-backend	linfa	k-means · DBSCAN · GMM · PCA (via boundary conversion)
hpo	hyperopt-rs · tpe	Bayesian / TPE hyperparameter search
automl	hyperopt-rs · model-selection-rs	automated preprocessing + model + HPO search with an auto-ensembled, deployable winner
diagnostics	regression-diagnostics	VIF · residual tests · influence · summary()
explain	shap-rs	SHAP values · permutation importance
calibration	core · plotters-statistical	probability calibration (Platt · isotonic) + reliability diagrams
anomaly	core · ndarray	outlier detection: Mahalanobis · kNN score (Isolation Forest as the ecosystem matures)
viz	plotters-statistical	ROC · calibration · residual · learning-curve charts
onnx	onnx-export-rs · tract	export trained pipelines · load & run ONNX
serve	axum · tokio	HTTP inference server + input validation
monitor	driftwatch · tracing	PSI / data & prediction drift · metrics endpoint
registry	core · serde	versioned model registry: pipeline + ONNX + metadata + reference distribution
timeseries	chronos-ts	ARIMA / auto-ARIMA forecasters · stationarity
incremental	incremental-rs	out-of-core partial_fit pipelines

FEATURE	CRATE	ADDS
python	pyo3 · numpy	Python package (<code>pip install millwright</code>): pandas / numpy / polars interop, Jupyter-ready

full = [every feature above] · default = ["smartcore-backend", "preprocessing", "model-selection"]

ROADMAP

Ship the spine, then light up features.

Build the contract and one backend first — a working `fit` / `predict` / `Pipeline` is the smallest thing that proves the design. Everything after is an adapter behind traits that already exist.

● PHASE 0 · THE SPINE

Frame, traits & the first backend

The `Frame` data model, the four traits, a `smartcore` adapter, and `Pipeline` composition.

ships → `fit` · `transform` · `predict` · `Pipeline`

○ PHASE 1 · PREP & SELECT

Preprocessing + cross-validation

Imputers, scalers, encoders, SMOTE as transformers; stratified CV and grid/random search over a whole pipeline — plus voting & bagging ensembles, with stacking riding the same CV engine.

ships → a `real`, `tunable`, `ensemble-ready` workflow

○ PHASE 2 · BACKENDS & HPO

linfa adapter + Bayesian search

Prove the boundary conversion with `linfa` clustering/PCA; add `TPE` / `hyperopt`-rs search behind the same search API.

ships → `two` backends, `one` contract

○ PHASE 3 · INSIGHT

Evaluation, diagnostics & explainability

Evaluation reports, regression-diagnostics, SHAP via `shap-rs`, and `plotters`-statistical report figures.

ships → `trust the model`, `not just run it`

○ PHASE 4 · PORTABILITY & PYTHON

ONNX artifact + the first bindings

Export any pipeline via `onnx-export-rs`; a `tract`-backed `InferenceModel`; and the first `pyo3` bindings over the now-stable trait API, so the Python package tracks the same core.

ships → `train once`; `run in Rust`, `Python`, or any `ONNX runtime`

○ PHASE 5 · OPERATIONS

Serving, monitoring & a registry

`axum` server, `driftwatch` monitoring, and a versioned model registry (artifact + metadata + reference distribution).

ships → `past where scikit-learn stops`

○ PHASE 6 · SPECIALIZED

Time series & out-of-core

`chronos-ts` forecasting pipelines and `incremental-rs` `partial_fit` — same contract, different data shapes.

ships → the long tail of real workloads

○ PHASE 7 · SYNTHESIS

AutoML — the framework, pointed at itself

Search preprocessing × model × hyperparameters × ensembling under a budget, seeded by **Profile** and scored by the CV engine — returning a deployable, ONNX-exportable winner and a leaderboard.

ships → auto-sklearn, but the output actually deploys

○ PHASE 8 · HARDEN → 1.0

Pin, prove, document

Exact-version pins on every engine, golden-output tests, a feature-matrix CI, and the existing guide re-cast as Millwright's tutorial.

ships → a framework you can bet on

— HONEST BOUNDARIES

What it is — and isn't.

Design commitments

- **Thin facade.** The core is Frame + four traits. Every engine is an adapter; the god-crate temptation is resisted by construction.
- **ONNX is the artifact.** The trained thing is portable and backend-agnostic — the training engine is an implementation detail by Phase 4.
- **The contract is stable; backends churn.** Commit to the traits in 0.1; let young crates evolve behind them. The framework becomes their stability layer.

Non-goals & risks

- **Not a numerics kernel.** No new linear algebra — it orchestrates proven implementations.
- **Not GPU/distributed in v1.** CPU + rayon parallelism; scale-out is a later story, flagged not faked.
- **Dependency maturity.** Ten young single-author crates underneath — mitigated by exact-version pins and a feature-matrix CI, but it is the real risk to own.
- **Conversion cost.** The two-ndarray bridge copies; measured, with zero-copy fast paths where layout allows.